

DESIGN PROCESS

PART I – Stateless Widgets



MEASURE TWICE, CUT ONCE

Acquire a **workflow** to layout UIs

<https://docs.flutter.dev/ui/layout/tutorial>

1

Diagram the layout

- *Identity the sections and the layout strategy*

2

Code the UI, sections by sections

- *Use Custom widgets to separate concerns*

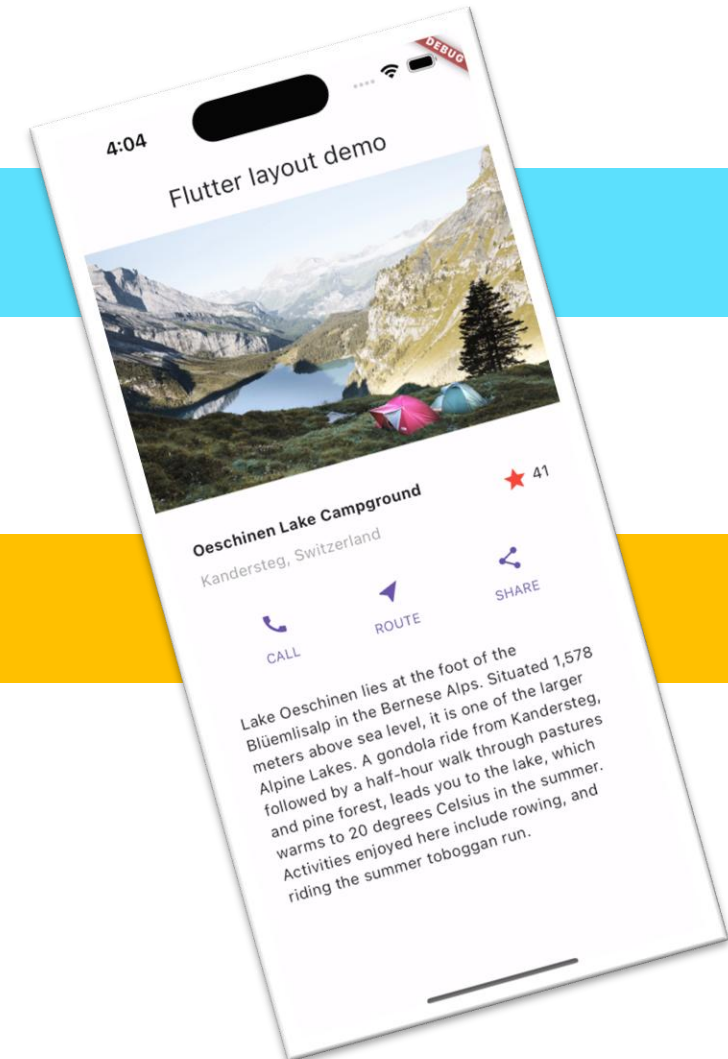
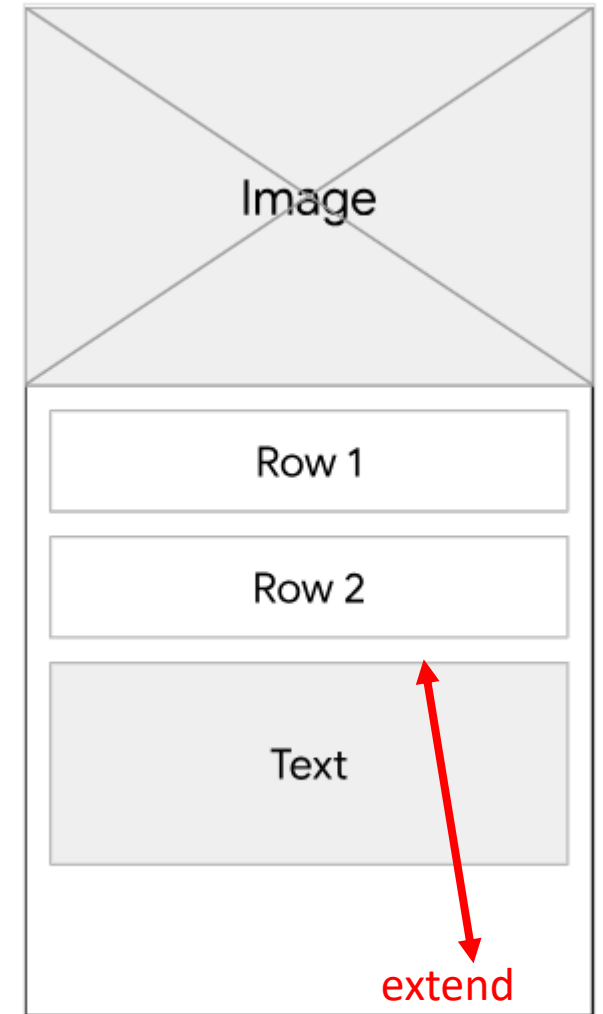
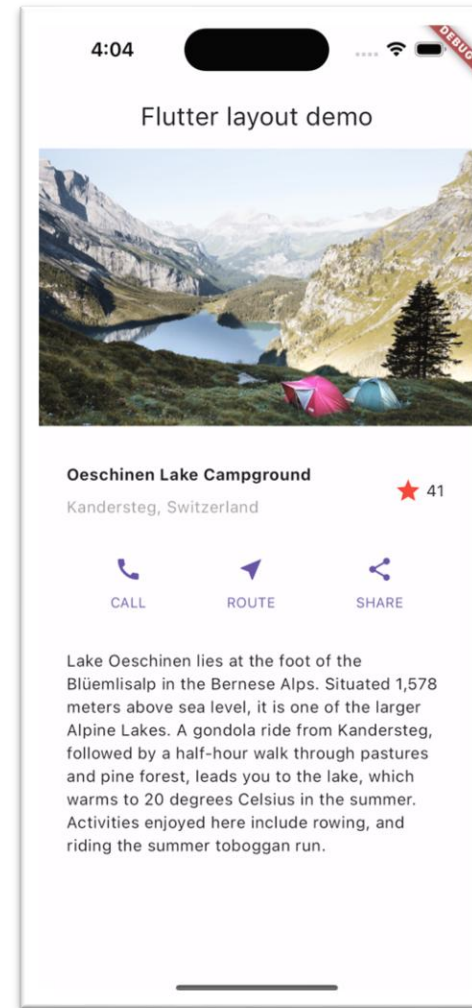


Diagram the layout

- Identify the **larger elements**.
- Ask yourself the basic questions
 - ✓ Can you identify the **rows** and **columns**?
 - ✓ Does the layout include a **grid**?
 - ✓ Are there **overlapping elements**?
 - ✓ Does the UI need **tabs**?
 - ✓ What do you need to **align, pad, or border**?
 - ✓ What do you need to **extend** ?



Major elements in the layout: image, row, row, and text block

Diagram the layout

- Diagram each section.

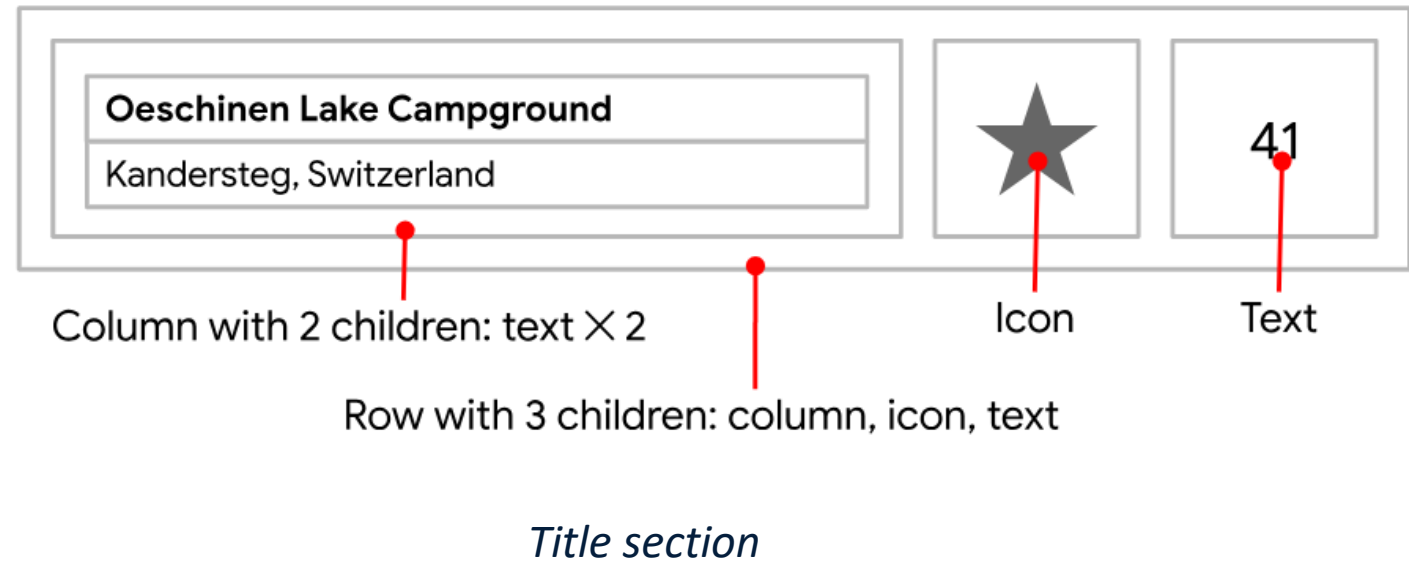
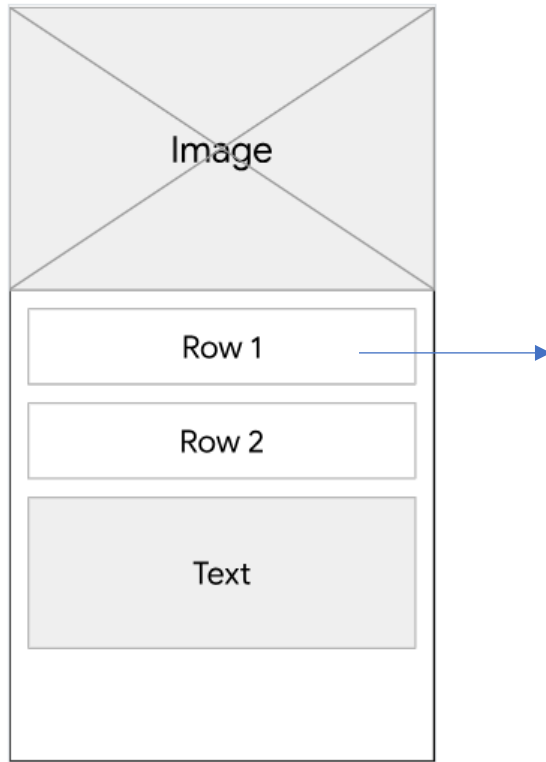
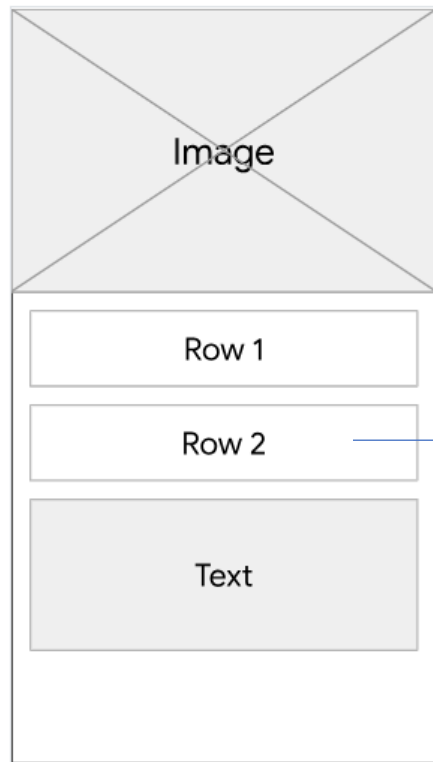


Diagram the layout

- Diagram each section.



Row with 3 children:
column, icon, text

Columns with
2 children: icon & text

Text Icon

Button section

Code the UI

- Diagram each section.



```
class TitleSection extends StatelessWidget {  
  const TitleSection({  
    super.key,  
    required this.name,  
    required this.location,  
  });  
  
  final String name;  
  final String location;
```

CHECK THE CORRECTION: HERE

```
@override  
Widget build(BuildContext context) {  
  return Padding(  
    padding: const EdgeInsets.all(32),  
    child: Row(  
      children: [  
        Expanded(  
          child: Column(  
            crossAxisAlignment: CrossAxisAlignment.start,  
            children: [  
              Padding(  
                padding: const EdgeInsets.only(bottom: 8),  
                child: Text(  
                  name,  
                  style: const TextStyle(fontWeight: FontWeight.bold),  
                ), // Text  
              ), // Padding  
              Text(  
                location,  
                style: TextStyle(color: Colors.grey[500]),  
              ), // Text  
            ],  
          ), // Column  
        ), // Expanded  
        Icon(  
          Icons.star, color: Colors.red[500],  
        ), // Icon  
        const Text('41'),  
      ],  
    ), // Row  
  ); // Padding  
}
```

SingleChildScrollView

Wrap the column in a scrollable area !

[MORE INFO](#)

```
body: Column(  
  children: [ ...],  
)
```



Overflow !

```
body:  
  SingleChildScrollView(  
    child: Column(  
      children: [ ...],  
    )  
  ),
```



Understand the **Unbounded height** error !

```
home: Scaffold(  
    body: SingleChildScrollView(  
        child: Column( ←  
            children: [  
                Expanded( ←  
                    child: Padding(  
                        child: Container(height: 200),  
                    ),  
                ),  
            ],  
        ),  
    ),  
)),
```

— EXCEPTION CAUGHT BY RENDERING LIBRARY
The following assertion was thrown during RenderFlex children layout:
RenderFlex children have non-zero flex

1 - If a Column is inside a **scrollable area**, its **height is unbounded**, meaning it can be **as tall as necessary**.

2 - However, when you use a **widget like Expanded inside the Column**, you're telling that widget to stretch and fill all the **available space**.

3 - since the parent doesn't have a set height, the **Expanded widget doesn't know how much space to fill !**

```

└─ EXCEPTION CAUGHT BY RENDERING LIBRARY ─┘
The following assertion was thrown during performLayout():
RenderFlex children have non-zero flex but incoming height constraints are unbounded.
When a column is in a parent that does not provide a finite height constraint, for example if it is
in a vertical scrollable, it will try to shrink-wrap its children along the vertical axis. Setting a
flex on a child (e.g. using Expanded) indicates that the child is to expand to fill the remaining
space in the vertical direction.
These two directives are mutually exclusive. If a parent is to shrink-wrap its child, the child
cannot simultaneously expand to fit its parent.
Consider setting mainAxisAlignment to MainAxisAlignment.min and using FlexFit.loose fits for the flexible
children (using Flexible rather than Expanded). This will allow the flexible children to size
themselves to less than the infinite remaining space they would otherwise be forced to take, and
then will cause the RenderFlex to shrink-wrap the children rather than expanding to fit the maximum
constraints provided by the parent.

```

SOLUTION !

```
runApp(  
  MaterialApp(  
    home: Scaffold(  
      body: SingleChildScrollView(  
        child: Column(  
          mainAxisAlignment: MainAxisAlignment.min, ←  
          children: [  
            Padding(  
              padding: const EdgeInsets.all(8.0),  
              child: Container(color:  Colors.green, height: 200),  
            ), // Padding  
            Flexible (  
              fit: FlexFit.loose, ←  
              child: Padding(  
                padding: const EdgeInsets.all(8.0),  
                child: Container(color:  Colors.blue, height: 200),  
              ), // Padding  
            ), // Flexible  
            Padding(  
              padding: const EdgeInsets.all(8.0),  
              child: Container(color:  Colors.red, height: 100),  
            ) // Padding  
          ],  
        ), // Column  
      ), // SingleChildScrollView  
    ), // Scaffold // MaterialApp  
  );  
}
```

Setting mainAxisAlignment to MainAxisAlignment.min, which will make the Column take only the space its children need.

Use Flexible instead of Expanded, and set FlexFit.loose, so the flexible children take only the space they need instead of trying to fill an undefined height.

Create Empty Space

Padding or *Sized Box*



```
ListView(  
  children: [  
    Padding(  
      padding: EdgeInsets.all(8.0),  
      child: Text("hello"),  
    ),  
  ],  
)
```



```
ListView(  
  children: [  
    SizedBox(height: 20),  
    Text("hello"),  
  ],  
)
```

Usually easier
to understand..

MOBILE DEVELOPMENT

W4-S2 – Stateful Widgets





Course Objectives



- ✓ Attach **event** handlers using **anonymous functions** (closures) in Dart.
- ✓ **Differentiate** between **Stateless** and **Stateful** Widgets.
- ✓ Explain the **lifecycle of a Stateful Widget** in Flutter.
- ✓ **Identify use cases** where Stateful Widgets are needed.
- ✓ Create a **Stateful Widget** in Flutter.
- ✓ Use the **setState()** method to update the UI dynamically.

A traffic light **has a state** with 3 values : red, orange, green





Handling Events

Observe and understand the bellow code

```
void onPress(){\n  print("hello");\n}\n\nrunApp(\n  MaterialApp(\n    home: Column(\n      children: [\n        TextButton(\n          onPressed: onPress,\n          child: const Text("Press me"),\n        ),\n      ],\n    ),\n  ),\n);
```

What does this line means?

Why this function is not called with () ?

What is the type of onPressed ?

Handling Events

Observe and understand the bellow code

```
void onPress() {  
    print("hello");  
}  
  
runApp(  
    MaterialApp(  
        home: Column(  
            children: [  
                TextButton(  
                    onPressed: onPress,  
                    child: const Text("Press me"),  
                ),  
            ],  
        ),  
    ),  
);
```

What does this line means?

*If the button is pressed,
the function **onPress**
will be called back*

Why this function is not
called with () ?

Because it is **NOT** a call
It s a **callback**

*A reference to a function
that should be called if we pressed
on the button*

What is the type of **onPressed** ? **VoidCallback**

*Which is the signature of callbacks that have no arguments
and return no data.*

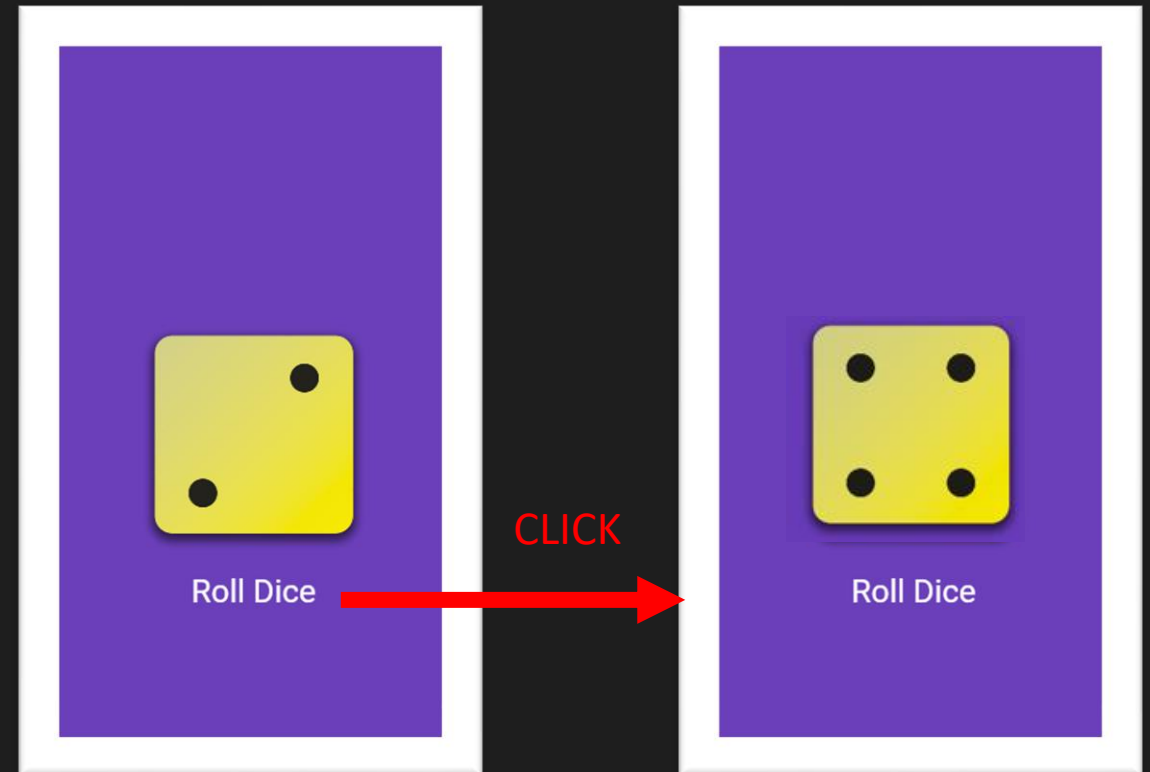
We want to change the dice image upon button pressed

What should we put on the rollDice() callback?

```
const String diceImage2 = 'assets/w4-s1/dice-2.png';  
const String diceImage4 = 'assets/w4-s1/dice-4.png';
```

```
String activeDiceImage = diceImage2;
```

```
class DiceRoller extends StatelessWidget {  
  const DiceRoller({super.key});  
  
  void rollDice() {  
    // Display the dice 4 !  
  }  
  
  @override  
  Widget build(context) {  
    return Column(  
      mainAxisAlignment: MainAxisAlignment.min,  
      children: [  
        Image.asset(  
          activeDiceImage,  
          width: 200,  
        ), // Image.asset  
        const SizedBox(height: 20),  
        TextButton(  
          onPressed: rollDice,  
          child: const Text('Roll Dice'),  
        ) // TextButton  
      ],  
    ); // Column  
  }  
}
```



Why is the widget not showing the updated value?

```
const String diceImage2 = 'assets/w4-s1/dice-2.png';  
const String diceImage4 = 'assets/w4-s1/dice-4.png';
```

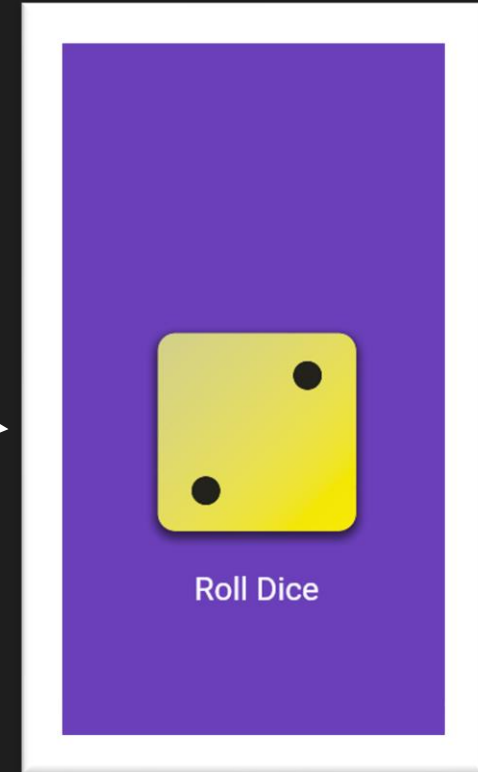
```
String activeDiceImage = diceImage2;
```

```
class DiceRoller extends StatelessWidget {  
  const DiceRoller({super.key});
```

```
  void rollDice() {  
    // Display the dice 4 !  
    activeDiceImage = diceImage4  
  }
```

```
  @override
```

```
  Widget build(BuildContext) {  
    return Column(  
      mainAxisAlignment: MainAxisAlignment.min,  
      children: [  
        Image.asset(  
          activeDiceImage,  
          width: 200,  
        ), // Image.asset  
        const SizedBox(height: 20),  
        TextButton(  
          onPressed: rollDice,  
          style: TextButton.styleFrom(  
            foregroundColor: Colors.white,  
            textStyle: const TextStyle(  
              fontSize: 28,  
            ), // TextStyle  
          ),  
          child: const Text('Roll Dice'),  
        ) // TextButton  
      ],  
    ); // Column  
  }
```



Because there is nothing telling the widget to rebuild itself when the "activeDiceImage" value changes.

A Stateless widget

will never **rebuild by itself**

- ✓ Stateless widgets **can't be changed once** they are built.
- ✓ These widgets are **immutable once they are built**
- ✓ Any change in data, widgets, icons, or variables do not change the state of the app or UI.
- ✓ They simply **override the build()** method and return a widget.



Let's refactor our widget to a
Stateful widget !

```
class DiceRoller extends StatefulWidget {
  const DiceRoller({super.key});

  @override
  State<DiceRoller> createState() {
    return _DiceRollerState();
  }
}
```

The stateful widget remains **unchanged over time**

The stateful widget is in charge of **creating the state object**

```
class _DiceRollerState extends State<DiceRoller> {
  String activeDiceImage = dice2;

  void rollDice() {
    setState(() {
      activeDiceImage = dice4;
    });
  }
}
```

The **state object** holds
 The active image
It's no longer final !

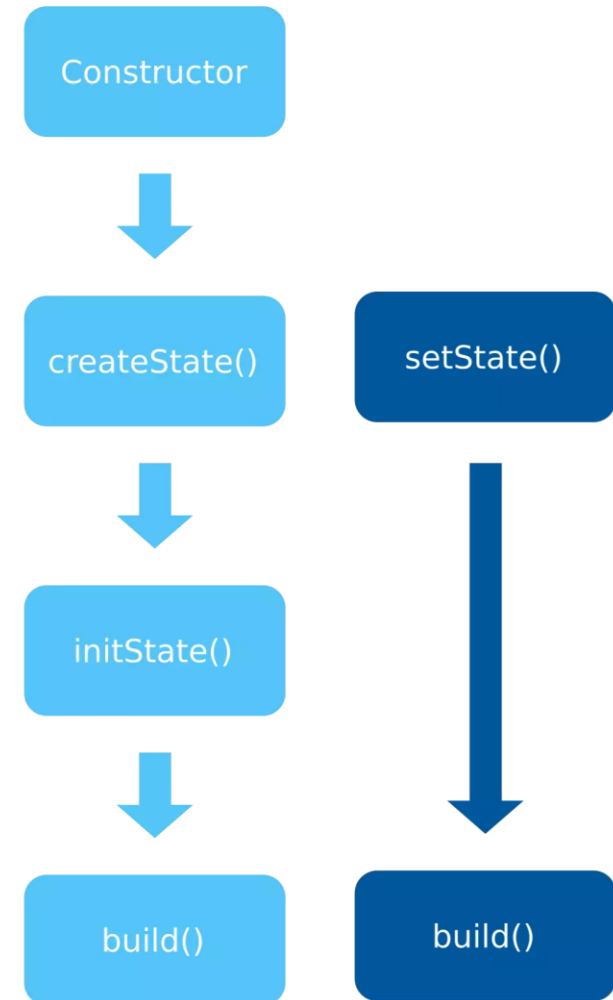
```
@override
Widget build(context) {
  return Column(
    mainAxisAlignment: MainAxisAlignment.min,
    children: [
      Image.asset(
        activeDiceImage,
        width: 200,
      ), // Image.asset
      const SizedBox(height: 20),
      TextButton(
        onPressed: rollDice,
        style: TextButton.styleFrom(
```

The state object is now in charge of **building the child widgets**

A Stateful widget

Will be rebuilt if **a state has changed**

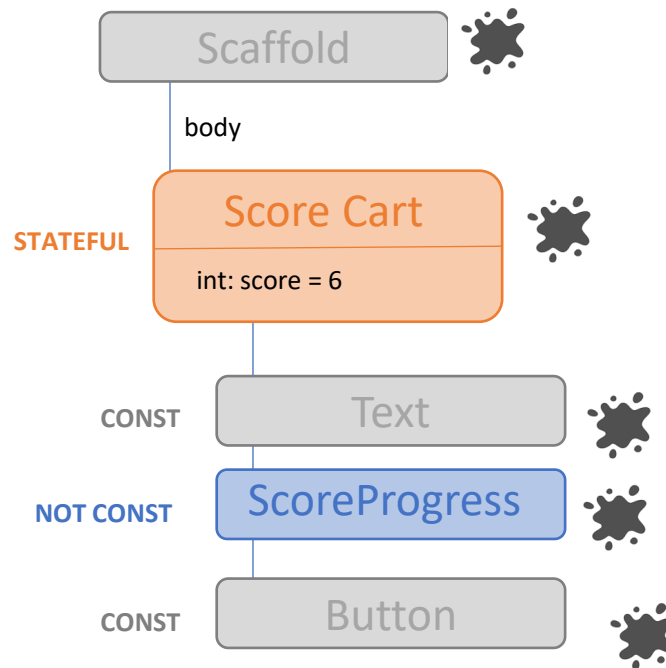
- ✓ Stateful widgets can rebuild its UI when its internal state changes
- ✓ It consists of two classes: a widget class and a state class.
- ✓ **The state is mutable**, and changes are triggered using the **setState()** method.
- ✓ It is used for **interactive elements** that need to be updated in case of dynamic changes.



When are Widget tagged as dirty ?

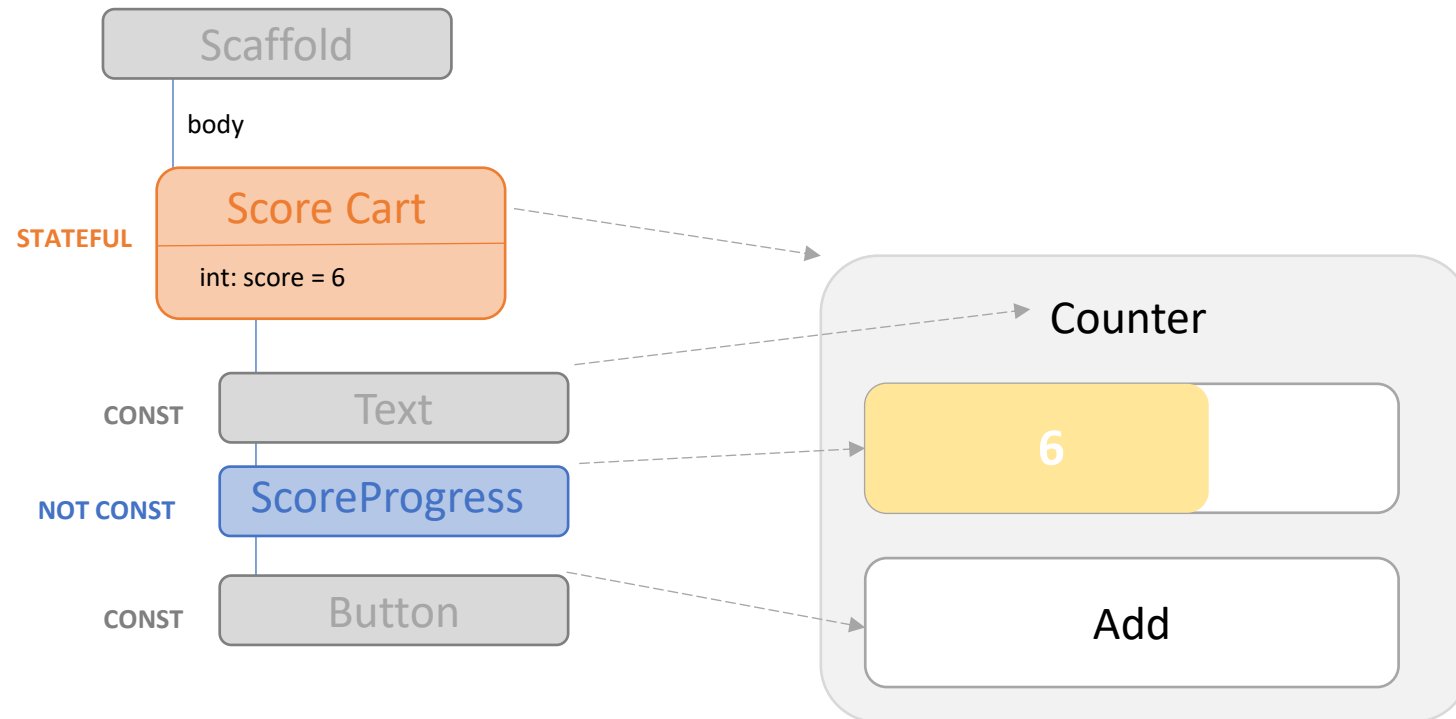
And need to be rebuild

- ✓ At the beggining all **widgets are dirty**: they need to be paint for the 1st time



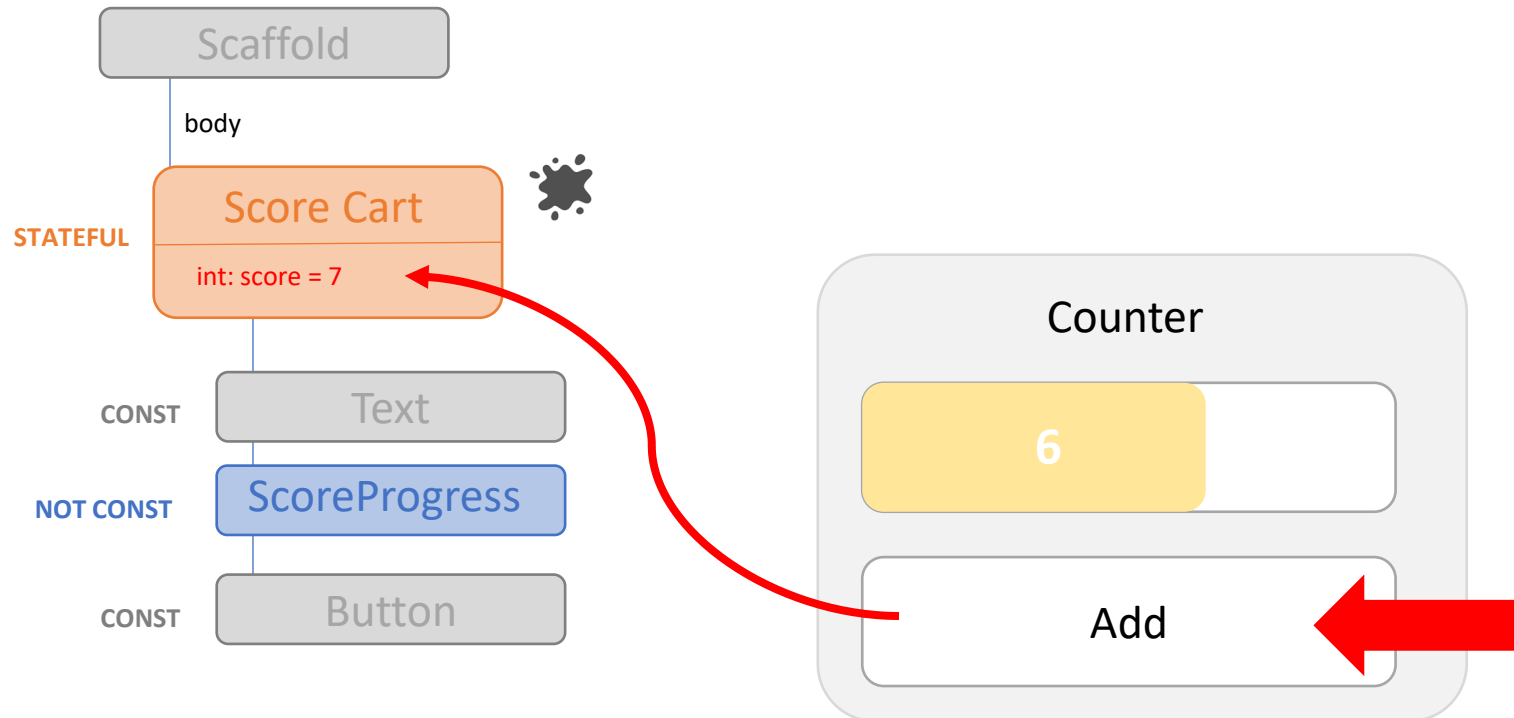
When Widget are **tagged as dirty** ? 🌟

- ✓ After building widgets, all widget are **now clean**



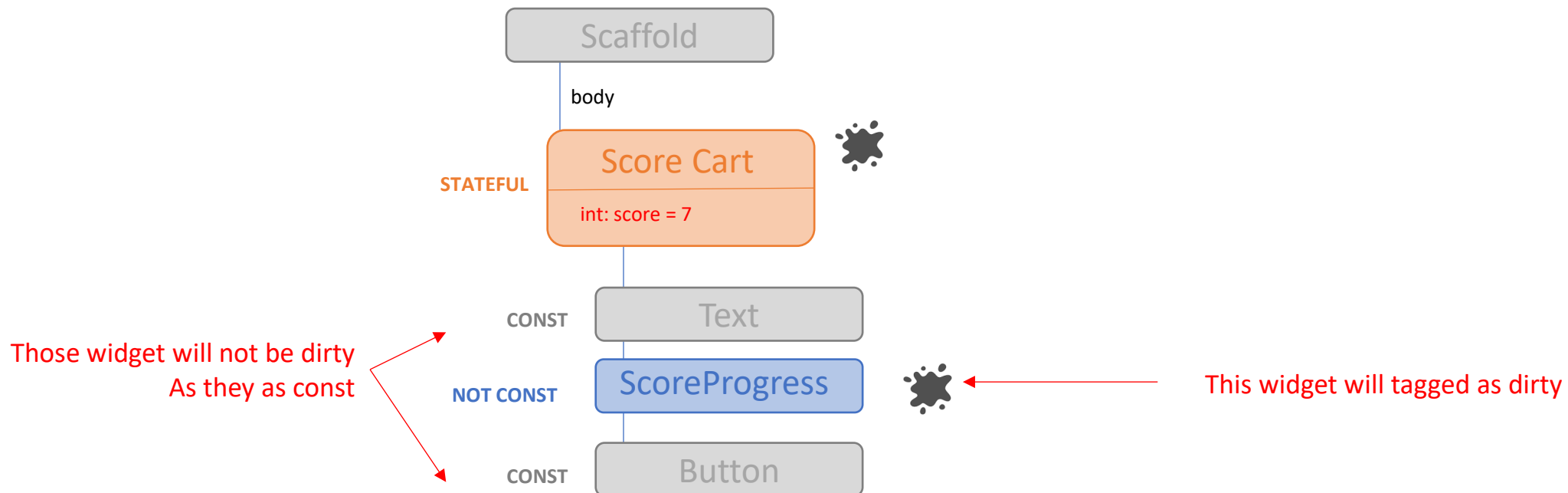
When Widget are tagged as dirty ?

- ✓ After clicking on Add, the **score state is increased**
- ✓ The **stateful widget is tagged dirty**



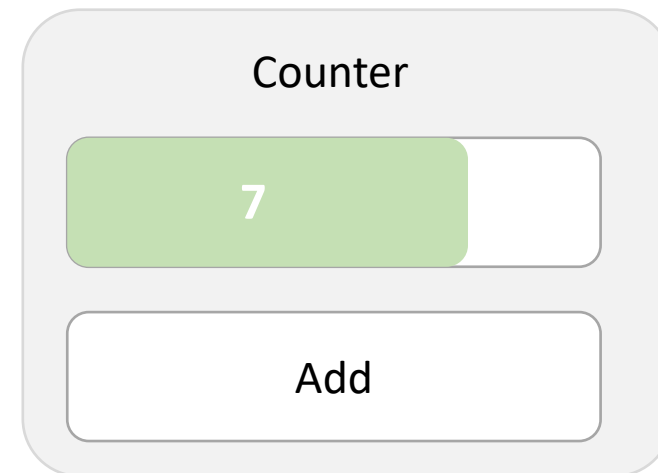
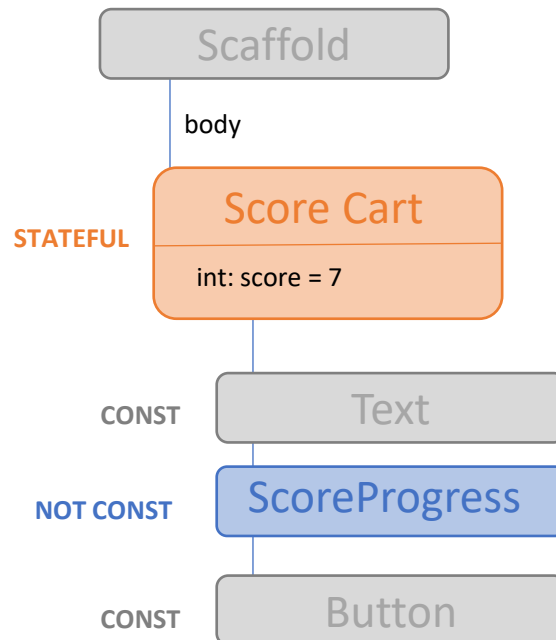
When Widget are tagged as dirty ?

- ✓ The children widgets will be marked as dirty too
- ✓ Expect the **const widgets**: as they never change !



When Widget are **tagged as dirty** ?

- ✓ The statefull widget and its sub widget **have been re-built**





Testing the **build()** calls

- ✓ Analyze when the build() is called on the 3 widgets (Counter, Label and Title)
? Why Counter and Label build() are called **and not the Title build** ?
- ✓ Remove the const on line 36 , and see the impact on the rebuild process

The screenshot displays an IDE with a Dart file named `main.dart` and a web browser showing the rendered application.

main.dart Code:

```
1 import 'package:flutter/material.dart';
2
3 Run Application
4 void main() {
5   runApp(
6     const MaterialApp(
7       title: 'My app',
8       home: Counter(),
9     ), // MaterialApp
10  );
11 }
12 class Counter extends StatefulWidget {
13   const Counter({
14     super.key,
15   });
16
17   @override
18   State<Counter> createState() => CounterState();
```

Browser View:

https://zeue06saeuf0.zapp.page/#/

Great counter

Current value: 3

Increment

Console Logs:

```
[17:18:33] Counter widget - rebuilding...
[17:18:33] Label widget - rebuilding...
```



Randomize the Dice !

- ✓ Get **the start code** and add the **assets** into you assets folders – Update your pubSpec !
- ✓ Change the code to **select a random dice** when clicking on the button
- ✓ *Extra : the roll should also work when clicking on the image*

TIP : DART RANDOM

TIP : GESTURE DETECTOR



When clicking on the button, the new dice should be randomly selected (1 to 6)